

LinkedIn Content Kit

5 Ready-to-Post Articles for hyper2kvm

Copy, customize, and post. Each article targets a different audience and angle.
Designed for maximum engagement on LinkedIn's algorithm.

Copy & Post | 5 Angles | Hashtags Included



Your Name

Title | Company

The Broadcom-VMware wake-up call: Why we migrated 350 VMs to KVM in 6 weeks.

When Broadcom acquired VMware, our vSphere renewal came in at 3x the previous year.

Three. Times.

We had 350 production VMs. Windows Server with SQL, RHEL with Java apps, Ubuntu with containers. The kind of environment that makes migration "impossible."

We did it anyway. Here's what actually happened:

Week 1-2: Assessment. Connected to vCenter, inventoried every VM. Classified them: 200 simple Linux, 100 medium complexity, 50 Windows with special needs.

Week 3-4: Pilot. Migrated 50 dev/test VMs using hyper2kvm. Every single one booted on first try. The tool automatically fixed fstab entries, rebuilt initramfs with virtio drivers, and removed VMware tools.

Week 5-6: Production. Batch-processed 300 VMs. 30-40 per night in maintenance windows. YAML configs made it reproducible. Windows VMs got automatic VirtIO driver injection. Zero customer-facing downtime.

The result?

\$1.2M saved per year. Same 3 admins (retrained in 2 weeks). No vendor lock-in. Full control of our infrastructure.

The hardest part wasn't the migration. It was deciding to start.

If you're staring at a VMware renewal that makes your eyes water, you have options. KVM is free. The tooling exists. The only question is how long you keep paying the tax.

hyper2kvm is open source (Apache 2.0). We used it, and now we're sharing it.

Link in comments.

#VMware #KVM #OpenSource #CloudMigration #Broadcom #Infrastructure #DevOps
#CostSavings #hyper2kvm

Posting tip: Post Tuesday-Thursday 8-10 AM your timezone. First line must hook — LinkedIn shows ~3 lines before "see more." Add the GitHub link as first comment, not in the post (better reach).



Your Name

Title | Company

Why VMs fail to boot after VMware-to-KVM migration (and how to fix it automatically)

Most migration tools do this:

1. Convert VMDK to qcow2
2. Boot it
3. Pray

Here's why that fails 70% of the time:

Problem #1: Storage drivers. VMware uses LSI Logic or PVSCSI controllers. KVM uses VirtIO. Without virtio_blk in initramfs, Linux can't see the boot disk. Windows blue-screens with INACCESSIBLE_BOOT_DEVICE.

Problem #2: fstab. VMware exports disks with /dev/sda1 device names. On KVM they become /dev/vda1. If fstab uses device names instead of UUIDs, boot hangs waiting for a device that doesn't exist.

Problem #3: GRUB. The root= parameter in GRUB points to the old device path. Wrong device = kernel panic.

Problem #4: Network. VMware VMXNET3 NIC disappears. Network manager configs reference a MAC address that no longer exists. VM boots but has no connectivity.

The fix? Don't just convert the disk. Fix the guest OS BEFORE first boot:

- Mount the disk image offline
- Inject virtio drivers into initramfs
- Convert fstab to UUID-based entries
- Update GRUB root= parameter
- Remove VMware NIC configs
- Trigger SELinux relabel

This is what hyper2kvm does automatically. It inspects the guest OS, applies the right fixes, and verifies boot success. 99.8% first-boot success rate.

One YAML config. One command. No manual intervention.

Open source (Apache 2.0). Link in comments.

[#Linux](#) [#KVM](#) [#VMware](#) [#SysAdmin](#) [#DevOps](#) [#Virtualization](#) [#OpenSource](#) [#Migration](#)
[#QEMU](#) [#LibGuestFS](#)

Posting tip: Technical posts get saved and shared by engineers. Include a code snippet image (YAML config screenshot) as the post image for higher engagement.



Your Name

Title | Company

**We moved 20 VMs from AWS to on-prem and saved \$900,000 over 3 years.
Here's the math.**

I know. "Cloud is cheaper." Everyone says it.

But here are the actual numbers from our AWS bill:

- 20 VMs (m5.2xlarge): \$5,500/month
- EBS storage (10TB): \$2,300/month
- Data egress (5TB/month): \$450/month
- Reserved instance premiums: \$1,200/month

Total: \$9,450/month = \$113,400/year

After repatriation to on-prem KVM:

- 2 Dell R750 servers: \$30,000 (one-time, amortized over 5 years = \$6,000/yr)
- NVMe storage: \$5,000 (one-time, amortized = \$1,000/yr)
- RHEL subscriptions: \$1,600/yr
- Power + cooling: \$3,600/yr

Total: \$12,200/year

Annual savings: \$101,200

3-year savings: \$303,600

Payback period: 4 months

But wait — we had 20 VMs on AWS. The real environment was 80 VMs across AWS and Azure.

Actual 3-year savings: \$916,000.

The migration itself? 4 weeks using hyper2kvm. Export from AWS, convert VHD/AMI to qcow2, deploy to local KVM. Automated fstab fixes, driver injection, boot verification.

Cloud makes sense for burst workloads and global distribution. For steady-state VMs? The math doesn't lie.

Tool is open source. Link in comments.

[#CloudRepatriation](#) [#AWS](#) [#Azure](#) [#CostOptimization](#) [#FinOps](#) [#Infrastructure](#) [#OnPrem](#)
[#KVM](#) [#OpenSource](#) [#TCO](#)

Posting tip: Financial posts with specific numbers get 3-5x more engagement. The "\$900,000" in the first line is the hook. Consider adding a simple comparison chart as the post image.



Your Name

Title | Company

We replaced VMware at 45 factory sites with K3s + KubeVirt. Zero on-site IT visits.

Here's how we managed it:

Each factory had an ESXi host running 3-5 SCADA/MES VMs. VMware licensing: \$10,000/site/year. Across 45 sites: \$450,000/year. Just for the hypervisor.

Our plan:

1. Convert all factory VMs at HQ using hyper2kvm (one weekend)
2. Ship pre-configured Intel NUCs (\$1,500 each) to factories
3. Local electrician plugs it in, connects network
4. K3s auto-joins fleet via WireGuard VPN
5. ArgoCD deploys VMs from Git repo
6. Dashboard shows all 45 sites from HQ

Time from plug-in to running VMs: 20 minutes. Per site.

No SSH needed. No on-site IT. No VMware license. No vCenter. No per-socket fees.

The entire fleet is managed via GitOps. Want to update a VM config across all 45 sites? Push to Git. ArgoCD handles the rest.

Total annual savings: \$450,000 in VMware licenses + \$200,000 in travel costs for on-site IT.

The KubeVirt + K3s combo is seriously underrated for edge computing. VMs as Kubernetes resources, managed with the same tools as containers. Best of both worlds.

Migration tool is open source (Apache 2.0): hyper2kvm. Link in comments.

[#EdgeComputing](#) [#KubeVirt](#) [#K3s](#) [#Kubernetes](#) [#Manufacturing](#) [#GitOps](#) [#IIoT](#) [#VMware](#)
[#OpenSource](#) [#Infrastructure](#)

Posting tip: Edge/manufacturing content reaches a different audience than typical DevOps posts. Tag specific industries. The "zero on-site visits" angle resonates strongly with operations leaders.



Your Name

Title | Company

We built a VMware migration tool with 1,273 tests, 10 cloud providers, and zero license fees. Here's why we open-sourced it.

Broadcom's VMware acquisition created a \$50B market in transition. 300,000+ organizations need to evaluate alternatives. Migration tools are the bottleneck.

The existing options?

- Red Hat MTV: Requires OpenShift (\$50K+/yr). KubeVirt only, no standalone KVM target.
- virt-v2v: CLI-only, no batch processing, no KubeVirt deploy, no TUI.
- AWS/Azure tools: Only migrate TO their cloud. Not helpful for repatriation.
- Carbonite: \$3K-10K/yr, Windows GUI, no automation.

We built hyper2kvm because we needed it ourselves. Then we realized everyone needs it.

What it does:

- Export from VMware, AWS, Azure, GCP, Hyper-V (10 providers via hypersdk)
- Convert any format: VMDK, OVA, VHD, RAW, AMI
- Offline guest fixes: fstab, initramfs, drivers, GRUB, SELinux
- Windows VirtIO injection (no BSOD)
- Deploy to KVM/libvirt OR KubeVirt/Kubernetes
- Interactive TUI (zkvm) + YAML automation
- 1,273 unit tests. Pre-flight checks. Boot verification.

Apache 2.0 — no catch, no upsell, no "community vs enterprise" split.

Why open source? Because vendor lock-in caused this problem in the first place.

The solution shouldn't create a new one.

Stars, issues, and PRs welcome. Link in comments.

[#OpenSource](#) [#VMware](#) [#KVM](#) [#Python](#) [#Go](#) [#DevOps](#) [#Migration](#) [#KubeVirt](#) [#Apache2](#)
[#CloudNative](#) [#hyper2kvm](#)

Posting tip: Open source announcement posts get strong developer engagement. Pin this as your featured post on LinkedIn. Include a screenshot of the zkvm TUI as the post image — visual posts get 2x reach.

Posting Strategy

Schedule (2 weeks):

Week 1:

- Monday: Post #5 (Open Source announcement — sets context)
- Wednesday: Post #2 (Technical how-to — builds credibility)
- Friday: Post #3 (Cloud cost story — financial hook)

Week 2:

- Tuesday: Post #1 (Broadcom story — thought leadership)
- Thursday: Post #4 (Edge computing — different audience)

Image strategy:

- Post #1: Before/after cost comparison chart
- Post #2: YAML config screenshot or terminal output
- Post #3: AWS bill vs on-prem cost infographic
- Post #4: Architecture diagram (K3s + KubeVirt at edge)
- Post #5: zkvm TUI screenshot

Engagement tactics:

- Put GitHub link in FIRST COMMENT (not in post — better algorithm reach)
- Reply to every comment within 1 hour (triggers algorithm boost)
- Ask a question at the end: "What's your VMware exit strategy?"
- Tag 2-3 relevant people (not spam — genuine connections)
- Repost each article 30 days later with "Update: X more VMs migrated"

Hashtag rules:

- Use 5-9 hashtags (LinkedIn sweet spot)
- Mix broad (#DevOps, #OpenSource) with niche (#KubeVirt, #VMware)
- Always include #hyper2kvm for brand tracking
- Put hashtags at the END, after content

Measuring success:

- Track: impressions, reactions, comments, profile views, GitHub stars
- Good post: 5K+ impressions, 50+ reactions, 10+ comments
- Great post: 20K+ impressions, 200+ reactions, 50+ comments
- If a post performs well, boost it with \$20-50 LinkedIn ad spend

